

ESTRUCTURAS DE CONTROL

Metodología de Programación Estructurada

Programación estructurada:

Metodología que consiste en escribir programas con las siguientes reglas:

- El programa tiene un **diseño modular**
- Los módulos son diseñados de **modo descendente**
- Cada módulo se codifica utilizando las **tres estructuras de control básicas**:
 - ✓ **Secuencia**
 - ✓ **Selección**
 - ✓ **Repetición**

Metodología de Programación Estructurada

Teorema fundamental de la PE (Böhm y Jacopini, 1966):

“Todo programa propio puede ser escrito usando solamente las estructuras de control básicas: secuencia, selección y repetición”

Programa propio:

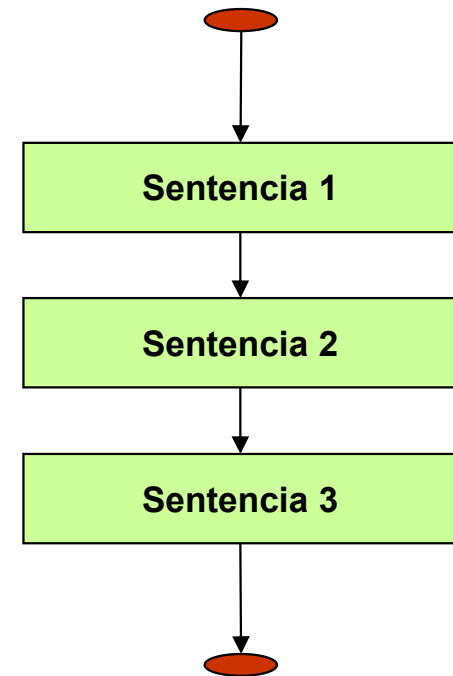
- ☐ Tiene un único punto de entrada y un único punto de salida.
- ☐ Existen caminos desde la entrada a la salida que se pueden seguir y que pasan por todas las partes del programa.
- ☐ Todas las instrucciones son ejecutables y no hay bucles infinitos.

ESTRUCTURAS DE CONTROL

- **Controlan el flujo de ejecución** de las acciones de un programa
- **Permiten combinar instrucciones o sentencias individuales** en una simple unidad lógica con un único punto de entrada y un único punto de salida
- Se clasifican en **tres tipos**:
 - ✓ Secuencia (Composición secuencial)
 - ✓ Selección (Composición alternativa), y
 - ✓ Repetición (Composición iterativa).

SECUENCIA

- ❑ Conjunto de sentencias que se ejecutan una luego de la otra (flujo secuencial).



- ❑ En C++: conjunto de sentencias que finalizan con ';' y están encerradas entre llaves.

```
{  
    sentencia-1;  
    sentencia-2;  
    .....  
    sentencia-n;  
}
```

SECUENCIA: Ejemplos

Ejemplo 1a: Se ingresan las notas de 3 parciales y se muestra la nota promedio.

```
int main() {  
    int n1, n2, n3;  
    cout << "Ingrese las notas de los 3 parciales: " << endl;  
    cin >> n1 >> n2 >> n3;  
    cout << endl << "La nota promedio es: " << (n1+n2+n3)/3;  
    return 0;  
}
```

```
Ingrese las notas de los 3 parciales:  
10  
7  
4  
  
La nota promedio es: 7
```

Ejemplo 2a: Se ingresan las coordenadas de un punto del plano y se informa la distancia al origen.

```
int main() {  
    float x,y;  
    cout << "Ingrese las coordenadas: " << endl;  
    cin >> x >> y;  
    cout << "La distancia de " << x << " y " << y << " al origen es: ";  
    cout << sqrt(x*x+y*y) ;  
    return 0;  
}
```

Usar
<cmath>

```
Ingrese las coordenadas:  
2.5  
4.7  
La distancia de 2.5 y 4.7 al origen es: 5.32353
```

BLOQUE

- Un bloque es una unidad de ejecución mayor que una sentencia, y permite agrupar una secuencia de sentencias como una unidad.
- Para ello, formaremos un bloque delimitando (encerrando) entre dos llaves la secuencia de sentencias que agrupa.
- Es posible anidar bloques, es decir, se pueden definir bloques dentro de otros bloques.

```
int main()
{
    <sentencia_1> ;
    <sentencia_2> ;
    {
        <sentencia_3> ;
        <sentencia_4> ;
        . . .
    }
    <sentencia_n> ;
}
```

Declaraciones Globales y Locales

Se pueden declarar identificadores en diferentes bloques, pudiendo entrar en conflicto unos con otros, por lo que debemos conocer las reglas utilizadas en el lenguaje C++ para decidir a qué identificador nos referimos en cada caso.

Hay dos clases de declaraciones: **globales y locales**.

- **Entidades globales** son aquellas que han sido definidas **fuera** de cualquier bloque. Su ámbito de visibilidad comprende desde el punto en el que se definen hasta el final del archivo. Se crean al principio del programa y se destruyen al finalizar éste.
- **Entidades locales** son aquellas que se definen **dentro** de un bloque. Su ámbito de visibilidad comprende desde el punto en el que se definen hasta el final de dicho bloque. Se crean en el punto donde se realiza la definición, y se destruyen al finalizar el bloque. Normalmente las variables se declararán locales, aunque también es posible declarar constantes localmente.

Declaraciones Globales y Locales

```
#include <iostream>
using namespace std;
const double EUR_PTS = 166.386; //Declaración de constante GLOBAL

int main() {
    cout << "Introduce la cantidad (en euros): ";
    double euros; //Declaración de variable LOCAL
    cin >> euros;
    double pesetas = euros * EUR_PTS; //Declaración de variable LOCAL
    cout << euros << " Euros equivalen a " << pesetas << " Pts" << endl;
    return 0;
}
```

```
Introduce la cantidad (en euros): 147
147 Euros equivalen a 24458.7 Pts
```

```
<< El programa ha finalizado: codigo de salida: 0 >>
<< Presione enter para cerrar esta ventana >>
```

Declaraciones Globales y Locales

- En caso de tener declaraciones de **diferentes entidades con el mismo identificador en diferentes niveles de anidamiento**, será visible la entidad declarada en el bloque más interno.
- La **declaración** más interna **oculta** a aquella más externa.

```
int main() {  
    int x= 3;  
    int z = x * 2;    // x es vble de tipo int con valor 3  
    {  
        double x = 5.0;  
        double n = x * 2;    // x es vble de tipo double con valor 5.0  
    }  
    int y = x + 4; // x es vble de tipo int con valor 3  
    return 0;  
}
```

No es una buena práctica de programación ocultar identificadores al redefinirlos en niveles de anidamiento más internos, ya que conduce a programas difíciles de leer y propensos a errores. **Se debe evitar esta práctica.**

Estructuras de Control de Selección

SELECCIÓN: SENTENCIA IF

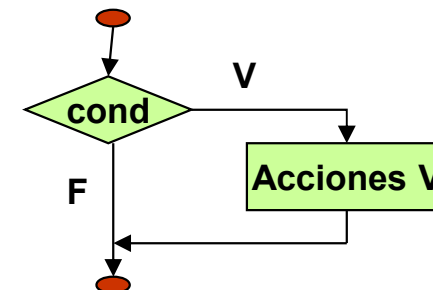
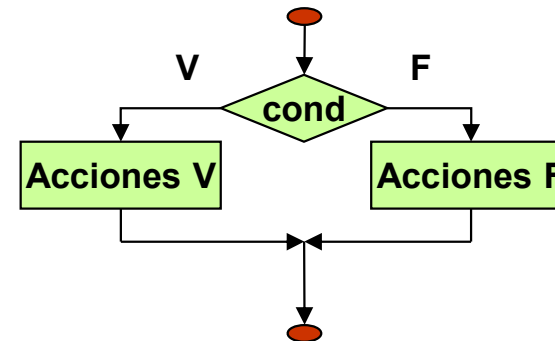
- ❑ **Selección completa**: Se evalúa una expresión booleana y según sea verdadera o falsa, se ejecuta una acción u otra; por cualquier caso la ejecución continúa con la sentencia siguiente al if.
- ❑ **Selección incompleta**: Sólo se realiza una acción en caso que la expresión booleana sea verdadera; no hay acción por el falso.
- En C++:

SELECCIÓN
COMPLETA

```
if (expresión)  
    acción_verdadero;  
else  
    acción_falso;
```

SELECCIÓN
INCOMPLETA

```
if (expresión)  
    acción_verdadero;
```



Las acciones pueden ser una sentencia que termina en “;” o un grupo de sentencias encerradas entre llaves.

SELECCIÓN INCOMPLETA: Ejemplos

Ejemplo 1b: Se ingresan las notas de 3 parciales y se debe indicar la nota final, siendo ésta el promedio ajustado (si está entre 6,3 y 6,5, se considera 6,5).

```
int main(){
    int n1, n2, n3;
    float prom;
    cout << "Ingrese las notas de los 3 parciales: " << endl;
    cin >> n1 >> n2 >> n3;
    prom = (float)(n1+n2+n3)/3;
    if (prom >= 6.3 && prom <= 6.5)
        prom = 6.5;
    cout << "La nota promedio es: " << prom;
    return 0;
}
```

```
Ingrese las notas de los 3 parciales:
10
7
4
La nota promedio es:7
```

```
Ingrese las notas de los 3 parciales:
10
5
4
La nota promedio es: 6.5
```

SELECCIÓN INCOMPLETA: Ejemplos

Ejemplo 2b: Se ingresan las coordenadas de un punto del plano y se debe informar si se encuentra en uno de los ejes coordenados.

```
int main() {  
    float x,y;  
    cout << "Ingrese las coordenadas: " << endl;  
    cin >> x >> y;  
    if ((x == 0) || (y==0))  
        cout << "El punto (" << x << "," << y << ") esta sobre un eje del sistema"  
        << endl;  
    return 0;  
}
```

```
Ingrese las coordenadas:  
2.7  
0  
El punto (2.7,0) esta sobre un eje del sistema
```

```
Ingrese las coordenadas:  
1.3  
6.8
```

SELECCIÓN COMPLETA: Ejemplos

Ejemplo 1c: Se ingresan las notas de 3 parciales y se debe indicar si ha aprobado (se aprueba con promedio mayor o igual a 6,5) o debe realizar un recuperatorio, y en ambos casos se muestra la nota.

```
int main(){
    int n1, n2, n3;
    float prom;
    cout << "Ingrese las notas de los 3 parciales: " << endl;
    cin >> n1 >> n2 >> n3;
    prom = (float)(n1+n2+n3)/3;
    if (prom >= 6.3 && prom <= 6.5)
        prom = 6.5;
    if (prom >= 6.5)
        cout << "Ha aprobado el curso" << endl;
    else
        cout << "Debe realizar un examen recuperatorio" << endl;
    cout << "Su nota promedio es: " << prom;
    return 0;
}
```

```
Ingrese las notas de los 3 parciales:
10
10
9
Ha aprobado el curso
Su nota promedio es: 9.66667
```

```
Ingrese las notas de los 3 parciales:
5
6
6
Debe realizar un examen recuperatorio
Su nota promedio es: 5.66667
```

SELECCIÓN COMPLETA: Ejemplos

Ejemplo 2c: Se ingresan las coordenadas de un punto del plano. Validar que no sea el origen y mostrar la distancia al origen.

```
int main() {  
    float x,y;  
    cout << "Ingrese las coordenadas: " << endl;  
    cin >> x >> y;  
    if (x == 0 && y==0)  
        cout << endl << "Incorrecto, ha ingresado el origen";  
    else  
        cout << endl << "La distancia de " << x << "," << y << " al origen es: "  
        << sqrt(x*x + y*y);  
    return 0;  
}
```

Ingrese las coordenadas:

1
0

La distancia de 1,0 al origen es: 1

Ingrese las coordenadas:

0
0

Incorrecto, ha ingresado el origen

SELECCIÓN ANIDADA

Permite evaluar varios if.

Una sentencia if es anidada cuando la acción de la rama verdadera y/o la rama falsa, **es también una sentencia if**.

- En C++:

```
if (exp1)
    acción 1;
else
    if (exp2)
        acción2; Se abre la rama falsa
    else
        acción3;
```

```
if (exp1)
    if (exp2)
        acción 1; Se abre la rama verdadera
    else
        acción 2;
else
    acción 3;
```

- El if exterior o los if interiores pueden ser incompletos. Ejemplos:

```
if (exp1) {
    if (exp2)
        acción2;
    else
        acción3;
}
```

if exterior incompleto

```
if (exp1) {
    if (exp2)
        acción 1;
}
else
    acción 3;
```

if interior incompleto

Importante: Las llaves se deben agregar para indicar al compilador C++ la asociación de los if-else.

SELECCIÓN ANIDADA: Asociación if-else

El compilador C++ siempre asocia un **else** con el **if** inmediatamente anterior (el que le precede). Si queremos que se realice otra asociación diferente, deberemos colocar llaves.

Ejemplo:

```
if (a > 5)
    if (b > 5)
        cout << "a y b son > 5" ;
else
    cout << "a es <= 5" ;
```

Este código parece indicar que si *a* es mayor que 5, se evaluará la instrucción if anidada y si es verdadera se mostrará la cadena "a y b son > 5". Pero, si *a* no es mayor que 5, parecería que se mostrará la cadena "a es <= 5" correspondiente a la instrucción else del if externo.

¡Cuidado! Esta instrucción if anidada **no se ejecuta como parece ser** dado que el compilador C++ en realidad interpreta la instrucción así:

```
if (a > 5)
    if (b > 5)
        cout << "a y b son > 5" ;
else
    cout << "a es <= 5" ;
```

El if externo es un if anidado incompleto. El if externo evalúa si *a* es mayor que 5 y si es verdadero evalúa si *b* también es mayor que 5.

Si la 2da condición es verdadera, se muestra la cadena "a y b son > 5".

Si la 2da condición es falsa se muestra la cadena "a es <= 5", aún cuando sabemos que *a* es mayor que 5.

SELECCIÓN ANIDADA: Asociación if-else

Para **forzar** la asociación if-else, debemos utilizar llaves:

```
if (a > 5) {  
    if (b > 5)  
        cout << "a y b son > 5" ;  
}  
else  
    cout << "a es <= 5" ;
```

Las llaves le indican al compilador C++ que la segunda instrucción if (es decir, el if anidado interno) se encuentra en el cuerpo del primer if, y que el **else** está asociado con el **primer if** (es decir, con el if externo).

SELECCIÓN ANIDADA: Ejemplos

Ejemplo 3: Se ingresa un número, se debe incrementar un contador general de números tratados (ContGral) y también se debe incrementar el contador correspondiente: ContPos, ContNul, ContNeg

¿Qué diferencia hay entre estas dos formas?

```
{  
    cin >> n;  
    if (n > 0)  
        ContPos ++;  
    else  
        if (n < 0)  
            ContNeg ++;  
        else  
            ContNul ++;  
    ContGral ++;  
}
```

```
cin >> n;  
if (n > 0)  
    ContPos ++;  
if (n < 0)  
    ContNeg ++;  
if (n==0)  
    ContNul ++;  
ContGral ++;
```

Las sangrías no son requeridas,
pero aportan claridad.

SELECCIÓN ANIDADA: Ejemplos

Ejemplo 1d: Se ingresan las notas de 2 parciales y se debe indicar si ha aprobado o debe realizar un recuperatorio (se aprueba con promedio mayor o igual a 6,5). En ambos casos se muestra la nota. Si debe recuperar, y en uno de los parciales sacó 7 o más, solo recupera la Parte no aprobada (A o B).

```
int main(){
    int n1, n2;
    float prom;
    cout << "Ingrese las notas de los 2 parciales: " << endl;
    cin >> n1 >> n2;
    prom = (float)(n1+n2)/2;
    if (prom >= 6.5)
        cout << "Ha aprobado el curso" << endl;
    else
        if (n1 >= 7)
            cout << endl << "Debe realizar un recuperatorio de la parte B";
        else
            if (n2 >= 7)
                cout << endl << "Debe realizar un recuperatorio de la parte A";
            else cout << endl << "Debe realizar un recuperatorio global";
    cout << endl << "Su nota promedio es: " << prom;
    return 0;
}
```

Ingrese las notas de los 2 parciales:

7
2

Debe realizar un recuperatorio de la parte B
Su nota promedio es: 4.5

SELECCIÓN ANIDADA: Ejemplos

- Ejemplo 1e: Idem 1d, pero si aprobó con promedio mayor a 7, se le informa que está becado para el siguiente curso.

```
int main() {
    int n1, n2;
    float prom;
    cout << "Ingrese las notas de los 2 parciales: " << endl;
    cin >> n1 >> n2;
    prom = (float)(n1+n2)/2;
    if (prom >= 6.5) {
        cout << "Ha aprobado el curso" << endl;
        if (prom > 7)
            cout << "Ademas esta becado para el curso siguiente" << endl;
    }
    else
        if (n1 >= 7)
            cout << endl << "Debe realizar un recuperatorio de la parte B";
        else
            if (n2 >= 7)
                cout << endl << "Debe realizar un recuperatorio de la parte A";
            else cout << endl << "Debe realizar un recuperatorio global";
    cout << endl << "Su nota promedio es: " << prom;
    return 0;
}
```

Ver que ocurre si
no se colocaran
las llaves !!!

```
Ingrese las notas de los 2 parciales:
9
8
Ha aprobado el curso
Ademas esta becado para el curso siguiente
Su nota promedio es: 8.5
```

SELECCIÓN ANIDADA: Ejemplos

- **Ejemplo 2d:** Dado un punto del plano informar sobre qué eje o en qué cuadrante está.

```
int main() {  
    float x,y;  
    cout << "Ingrese las coordenadas: " << endl;  
    cin >> x >> y;  
    if (x == 0)  
        cout << endl << "El punto " << x << "," << y << " esta sobre el eje y";  
    else if (y == 0)  
        cout << endl << "El punto " << x << "," << y << " esta sobre el eje x";  
    else if (x > 0 && y > 0)  
        cout << endl << "El punto " << x << "," << y << " esta en el 1er cuadrante";  
    else if (x > 0 && y < 0)  
        cout << endl << "El punto " << x << "," << y << " esta en el 4to cuadrante";  
    else if (x < 0 && y > 0)  
        cout << endl << "El punto " << x << "," << y << " esta en el 2do cuadrante";  
    else  
        cout << endl << "El punto " << x << "," << y << " esta en el 3er cuadrante";  
    return 0;  
}
```


Comparación de if anidados y secuencias de if

- El ejemplo 2d usa una única sentencia if anidada.
- Otra alternativa es usar una **secuencia** de if:

```
int main() {  
    float x,y;  
    cout << "Ingrese las coordenadas: " << endl;  
    cin >> x >> y;  
    if (x == 0)  
        cout << endl << "El punto " << x << "," << y << " esta sobre el eje y";  
    if (y == 0)  
        cout << endl << "El punto " << x << "," << y << " esta sobre el eje x";  
    if (x > 0 && y > 0)  
        cout << endl << "El punto " << x << "," << y << " esta en el 1er cuadrante";  
    if (x > 0 && y < 0)  
        cout << endl << "El punto " << x << "," << y << " esta en el 4to cuadrante";  
    if (x < 0 && y > 0)  
        cout << endl << "El punto " << x << "," << y << " esta en el 2do cuadrante";  
    if (x < 0 && y < 0)  
        cout << endl << "El punto " << x << "," << y << " esta en el 3er cuadrante";  
    return 0;  
}
```

- Ambas son lógicamente equivalentes, pero ésta es **menos EFICIENTE**, ya que si alguna de las condiciones es verdadera, las siguientes condiciones se evalúan igualmente (para cada punto se realizan entre 1 y 6 evaluaciones).

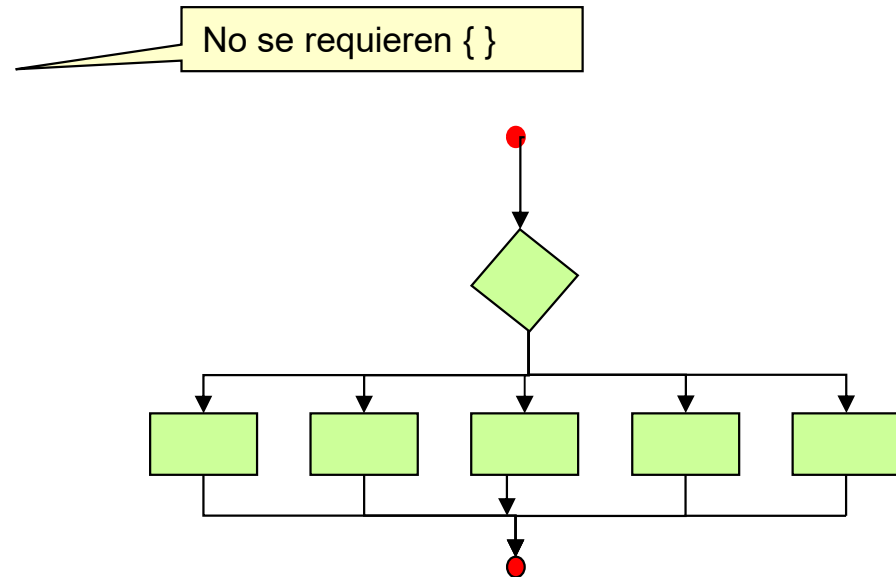
SELECCIÓN MÚLTIPLE: Sentencia SWITCH

- Se emplea para los casos en que se debe **seleccionar una de entre múltiples alternativas**, en forma exclusiva, basada en el valor de una variable o expresión denominada **selector o expresión de control**, la cual sólo puede ser **de tipo int o char**.
- El mecanismo se completa con la sentencia “**break**” que hace que siga la ejecución en la siguiente sentencia, dentro del programa, en este caso la sentencia siguiente al switch.
- El switch es muy útil para transformaciones de **datos para salida** y para el **manejo de menú** en las interfases de los programas.

Sentencia SWITCH

- En C++:

```
switch (selector) {  
    case etiqueta_1:  
        sentencias;  
        break;  
  
    case etiqueta_2:  
        sentencias;  
        break;  
  
    .....  
  
    default : /* opcional */  
        sentencias;  
        break;  
}
```



- Cada etiqueta es un **valor único, constante** (int o char) y debe tener un valor **diferente de los otros**.
- Se van evaluando las etiquetas y cuando alguna coincide con el valor del selector se ejecutan todas las sentencias asociadas.
- Si el valor del selector no es igual a ninguna etiqueta, no se ejecuta ninguna opción, a menos que se use el default.

Sentencia SWITCH

- Ejemplo 3: Mostrar la carrera de UTN teniendo los valores K, M, C, E, I en la variable carrera de tipo char.

```
int main() {  
    char carrera;  
    cout << "Ingrese carrera (K, M, C, E o I): ";  
    cin >> carrera;  
    switch (carrera) {  
        case 'K':  
            cout << " Ing. en Sistemas de Informacion ";  
            break;  
        case 'M':  
            cout << " Ing. Mecanica ";  
            break;  
        case 'C':  
            cout << " Ing. Civil ";  
            break;  
        case 'E':  
            cout << " Ing. Electrica";  
            break;  
        case 'I':  
            cout << " Ing. Industrial ";  
            break;  
        default:  
            cout << "No existe esa carrera";  
    }  
    return 0;  
}
```

```
Ingrese carrera (K, M, C, E o I): M  
Ing. Mecanica
```

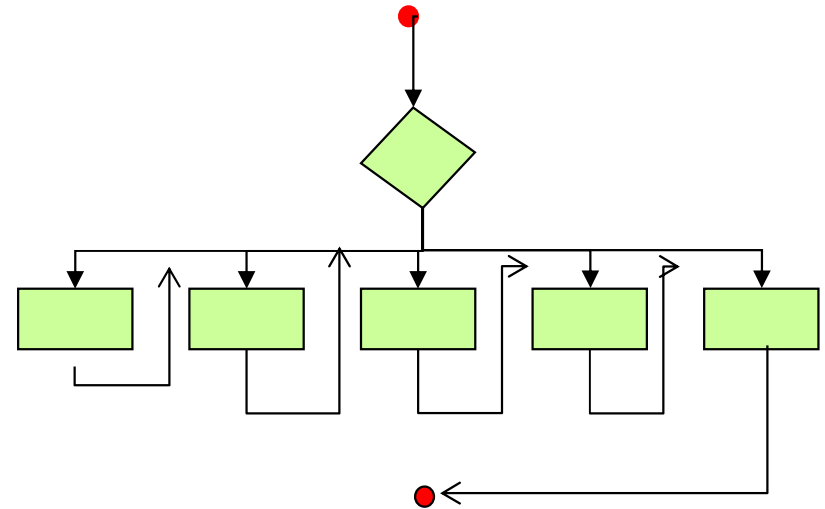
```
Ingrese carrera (K, M, C, E o I): L  
No existe esa carrera
```

Sentencia SWITCH (sin break)

- Tiene un **comportamiento en cascada**.

- En C++:

```
switch (selector) {  
    case etiqueta_1:  
        sentencias;  
    case etiqueta_2:  
        sentencias;  
    .....  
    default : /* opcional */  
              sentencias;  
}
```



- Se van evaluando las etiquetas y cuando el valor del selector es igual a una de las etiquetas, se ejecutan todas las sentencias a partir de la que corresponde a ese selector.

Sentencia SWITCH (sin break)

- Ejemplo 4: En una asignatura hay que realizar 4 evaluaciones que son correlativas. El estado de un alumno se maneja con un natural de 0 a 4 según las actividades que ya ha realizado. Se deben mostrar los mensajes correspondientes para un alumno conociendo su estado.

```
int main() {  
    int estado;  
    cout << "Ingrese valor correspondiente al estado del alumno: ";  
    cin >> estado;  
    switch (estado) {  
        case 0:  
            cout << "Debe realizar la Evaluación inicial" << endl;  
        case 1:  
            cout << "Debe realizar el TP1" << endl;  
        case 2:  
            cout << "Debe realizar el Parcial 1" << endl;  
        case 3:  
            cout << "Debe realizar el TP2" << endl;  
        case 4:  
            cout << "Debe realizar el Parcial 2" << endl;  
    }  
    return 0;  
}
```

- Si la variable *estado* tiene el valor 2 ➡
Ingrese valor correspondiente al estado del alumno: 2
Debe realizar el Parcial 1
Debe realizar el TP2
Debe realizar el Parcial 2

Sentencia SWITCH (con varios case)

- Está permitido **tener varias expresiones case** en una alternativa dada dentro de la sentencia switch, si se necesita.
- Ejemplo 5: Dado un valor dígito, indicar si es mayor, igual o menor que 3.

```
int main(){
    int digito;
    cout << "Ingrese valor digito: ";
    cin >> digito;
    switch (digito){
        case 0:
        case 1:
        case 2:
            cout << "Es menor que 3";
            break;
        case 3:
            cout << "Es igual a 3";
            break;
        default:
            cout << "Es mayor que 3" << endl;
    }
    return 0;
}
```

```
int main(){
    int digito;
    cout << "Ingrese valor digito: ";
    cin >> digito;
    switch (digito){
        case 0: case 1: case 2:
            cout << "Es menor que 3";
            break;
        case 3:
            cout << "Es igual a 3";
            break;
        default:
            cout << "Es mayor que 3" << endl;
    }
    return 0;
}
```

```
Ingrese valor digito: 1
Es menor que 3
```

```
Ingrese valor digito: 3
Es igual a 3
```

```
Ingrese valor digito: 8
Es mayor que 3
```

Errores frecuentes de programación

- Usar `=` en lugar de `==` en las condiciones.
- Confundir la asociación de `else` con el `if` correspondiente (solución: **encerrar entre llaves**).
- Que no haya concordancia entre el tipo del selector y el tipo de las etiquetas, en el `switch`.
- Omitir el `break` cuando sea requerido.

Para profundizar en el tema:

❑ Capítulo 4

Libro: “Fundamentos de Programación con el Lenguaje de Programación C++” – Benjumea-Roldan

❑ Capítulo 4

**Libro: “C++ para ingeniería y ciencias”, 2da Edición.
Gary J. Bronson.**

❑ Capítulo 4

**Libro: “Cómo programar en C++” – Harvey M. Deitel y
Paul J. Deitel**

❑ Capítulos 5 y 9

Libro: “Programación y resolución de problemas con C++” – Nell Dale - Chip Weems.